

Audit technique et architecture cible

Your Car Your Way

Applications web internationales de location de voitures

Sommaire

1. Introduction et méthode d'audit
 2. Synthèse de l'architecture existante
 3. Forces identifiées
 4. Faiblesses identifiées
 5. Contraintes techniques et métier à respecter
 6. Audit par critères : maintenabilité, performance, évolutivité, sécurité et disponibilité
 7. Impacts pour la proposition d'architecture
 8. Conclusion de l'audit
 9. Proposition d'architecture cible
 10. Spécifications techniques alignées avec les user stories
 11. Architecture applicative cible
 12. Modèle de données
 13. Intégration des composants tiers
 14. Justification des choix technologiques
 15. Sécurité, accessibilité et impact écologique
 16. Synthèse finale
- Références techniques utilisées

1. Introduction et méthode d'audit

Ce document présente l'audit de l'existant puis l'architecture cible proposée.

L'audit compare l'existant aux besoins du cahier des charges : application internationale, API agence, sécurité, accessibilité, disponibilité et règles locales. Les choix proposés répondent directement aux écarts constatés.

1.1 Critères retenus

Critère	Définition utilisée dans l'audit	Indicateurs observés
Maintenabilité	Capacité à corriger, faire évoluer, tester et déployer l'application sans coût excessif ni duplication incontrôlée.	Duplication de code, divergence des règles, taux de succès des déploiements, durée de stabilisation post-release.
Performance	Capacité à traiter les parcours critiques avec un temps de réponse acceptable, y compris lors des pics saisonniers.	Charge maximale sans dégradation, taux d'erreur en période de vacances, instabilité en charge.
Évolutivité	Capacité à ajouter des pays, règles locales, fonctionnalités et volumes d'utilisateurs sans multiplier les applications ni les modèles de données.	Monolithes, bases séparées, absence de modèle commun, API non unifiées.
Sécurité et confidentialité	Capacité à protéger comptes, données personnelles, flux, secrets et dépendances, en cohérence avec les exigences RGPD et paiement externalisé.	Hashage des mots de passe, TLS, stockage des secrets, vulnérabilités connues.
Disponibilité et résilience	Capacité du service à rester accessible, à récupérer après incident et à restaurer les données de manière maîtrisée.	Taux de disponibilité, MTTR, redondance, tests de restauration, sauvegardes.
Interopérabilité et données	Capacité à exposer des API cohérentes aux applications d'agence et à partager une donnée fiable entre pays.	API limitées, hétérogénéité des schémas, échanges manuels ou inexistants.

2. Synthèse de l'architecture existante

L'existant repose sur plusieurs applications nationales. Les technologies, les bases de données et les pratiques d'exploitation diffèrent selon les pays.

Périmètre	Technologies et hébergement	Constat d'audit
France / Allemagne / Espagne / Italie	Java EE, JSP/JSF, serveurs OVH, monolithes hérités, déploiements manuels.	Base fonctionnelle riche, mais vieillissante et dérivée par copie/adaptation, ce qui crée une divergence progressive.
Royaume-Uni	PHP Laravel, AWS EC2, application issue d'un rachat.	Application plus récente mais isolée, avec modèle de données et règles de réservation fortement différents.
Canada	React, Node.js, AWS, backend encore monolithique.	Meilleure expérience utilisateur et première tentative d'unification visuelle, mais limitée au pays.
États-Unis	Angular, Spring Boot, Azure App Services / Containers.	Stack la plus moderne et seule application containerisée, mais non généralisée et base non redondante.

2.1 Schéma simplifié de l'existant

Le schéma ci-dessous représente la fragmentation actuelle : plusieurs applications monolithiques, des bases de données séparées et des intégrations d'agence non homogènes.

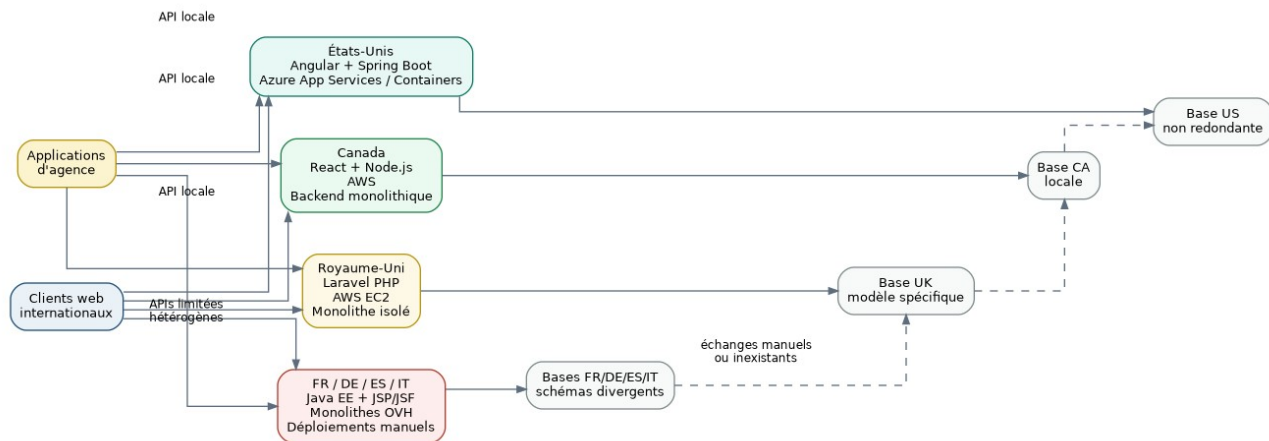


Figure 1 - Vue simplifiée de l'architecture existante

2.2 Données techniques chiffrées

Indicateur	FR/DE/ES/IT	UK	CA	US
Disponibilité moyenne sur 12 mois	97,2 %	98,6 %	98,1 %	98,9 %
MTTR moyen	≈ 2 h 45	≈ 1 h 10 (cloud)	≈ 1 h 10 (cloud)	≈ 1 h 10 (cloud)
Succès des déploiements	82 %	91 % (cloud)	91 % (cloud)	91 % (cloud)
Stabilisation post-release	3,4 jours	1,7 jour (cloud)	1,7 jour (cloud)	1,7 jour (cloud)
Dépendances vulnérables	FR 41 %, DE/ES/IT 35 à 40 %	18 %	22 %	11 %
Charge max sans dégradation	≈ 150 req/s	≈ 250 req/s	≈ 300 req/s	≈ 350 req/s
Erreur en pics saisonniers	jusqu'à 4 %	1,5 %	1,5 %	0,8 %
Backups	Manuels, 1 fois/jour, restauration non testée	Snapshots quotidiens, tests non réguliers	Snapshots quotidiens, tests non réguliers	Automatisés, restauration testée tous les 90 jours

3. Forces identifiées

L'existant n'est pas uniquement une dette technique. Il contient des acquis utiles pour cadrer la future architecture, à condition de distinguer les acquis métier des choix techniques à risque.

Force	Justification observée	Limite associée
Couverture fonctionnelle métier déjà éprouvée	Les applications existantes couvrent l'authentification, le catalogue, la réservation et le paiement, notamment sur le socle historique FR/DE/ES/IT.	Cette richesse est dispersée et parfois implémentée différemment selon les pays.
Connaissance des règles locales	Les applications nationales intègrent des règles de réservation et comportements spécifiques aux pays.	Ces règles sont codées localement, ce qui rend leur harmonisation difficile.
Expériences techniques variées	Laravel, React/Node.js, Angular/Spring Boot, AWS et Azure ont déjà été expérimentés.	La diversité n'a pas été gouvernée par une stratégie commune.
Amélioration UX au Canada	Le Canada a apporté une meilleure expérience utilisateur et une première unification visuelle.	L'amélioration reste locale et ne résout pas la fragmentation des données.
Maturité relative de la stack US	Les États-Unis ont la meilleure disponibilité, la meilleure capacité de charge, le plus faible taux de vulnérabilités et la seule application containerisée.	Le projet n'a pas été généralisé et sa base de données n'est pas redondante.
Pratiques de sécurité modernes dans certains pays	UK utilise bcrypt, CA argon2id, US bcrypt avec strength 12 ; KeyVault est utilisé partiellement côté US.	Les pratiques restent hétérogènes et insuffisamment généralisées.

Présence du cloud pour UK/CA/US	AWS et Azure offrent une base plus récente que les environnements OVH historiques.	La redondance, la rotation des secrets et les tests de restauration ne sont pas homogènes.
---------------------------------	--	--

4. Faiblesses identifiées

Synthèse

Les faiblesses principales sont la fragmentation par pays, la dette de sécurité, l'absence de modèle de données commun, la faible automatisation des déploiements historiques et la capacité insuffisante à absorber les pics saisonniers.

Domaine	Faiblesse constatée	Conséquence technique et métier
Architecture applicative	Architecture dominante composée de monolithes web, sans microservices et sans stratégie d'unification.	Les évolutions sont couplées, les impacts sont difficiles à isoler et la réutilisation inter-pays reste faible.
Maintenabilité	FR/DE/ES/IT reposent sur du code copié/adapté, avec divergence progressive.	Les corrections doivent être reproduites, les règles métier peuvent diverger et le risque de régression augmente.
Données	Chaque pays possède sa propre base avec des schémas divergents.	La donnée client/réservation n'est pas fiable à l'échelle internationale et le partage reste manuel ou inexistant.
API	APIs limitées, hétérogènes et non unifiées.	Les applications d'agence ne peuvent pas disposer d'un accès CRUD cohérent et sécurisé aux domaines métier.
Déploiement	Déploiements manuels sur FR/DE/ES/IT avec 82 % de succès.	Risque d'erreurs humaines, délais de mise en production plus longs, stabilisation post-release de 3,4 jours.
Sécurité	SHA-1 sur FR/DE/ES/IT, TLS 1.0 encore présent sur FR et IT, secrets dans fichiers de configuration, vulnérabilités connues élevées.	Risque accru sur comptes clients, données personnelles, flux de paiement et conformité.
Disponibilité	Absence de réplication applicative FR/DE/ES/IT, réplication partielle UK/CA, base US non redondante.	Le service ne garantit pas une continuité robuste sur les parcours critiques.
Sauvegarde et reprise	Restauration non testée sur FR/DE/ES/IT, tests non réguliers sur UK/CA.	La restauration après incident majeur reste incertaine.
Performance	Capacité FR/DE/ES/IT limitée à environ 150 requêtes/seconde, jusqu'à 4 % d'erreurs en pics saisonniers.	Risque de perte de réservation et de dégradation de confiance client lors des périodes à forte demande.

5. Contraintes techniques et métier à respecter

Les contraintes ci-dessous ne sont pas des solutions. Elles cadrent ce que la future architecture devra rendre possible pour satisfaire les exigences consolidées et éviter de reproduire les limites de l'existant.

Contrainte	Origine dans les besoins	Impact sur l'audit de l'existant
Application internationale unifiée	Remplacer les applications nationales par une application client commune.	L'existant par pays ne valide pas l'unification : il fragmente parcours, code et données.
Règles locales paramétrables	Conserver les règles pays : langue, devise, formats, fuseaux horaires, règles de réservation.	L'existant prouve l'existence de règles locales mais les implémente de manière dispersée.
API agence CRUD sécurisée	Les applications d'agence doivent consulter et modifier les domaines utilisateur, réservation, agence, offre et statuts.	Les APIs actuelles sont trop limitées et hétérogènes pour valider ce besoin.

Païement externalisé	Aucune donnée de carte bancaire ne doit être stockée dans le SI.	L'audit doit vérifier que l'architecture ne propage pas les risques de paiement dans les monolithes existants.
RGPD et suppression/anonymisation	Minimisation des données, droits d'accès, rectification et suppression.	Les bases séparées compliquent l'application homogène de ces exigences.
Sécurité moderne	TLS récent, hashage moderne, secrets hors code et logs centralisés dans ELK.	L'existant invalide partiellement ce critère, surtout sur FR/DE/ES/IT.
Accessibilité RGAA/WCAG	Parcours critiques accessibles clavier, lecteurs d'écran, erreurs explicites.	Les technologies anciennes JSP/JSF peuvent augmenter la dette d'accessibilité si elles ne sont pas auditées précisément.
Performance en pics saisonniers	Le parcours de réservation doit rester utilisable pendant les périodes de vacances.	Les erreurs jusqu'à 4 % et les limites de charge invalident ce critère pour le socle historique.
Éco-conception fonctionnelle	Pages utiles, chargement maîtrisé, absence de médias superflus.	L'hétérogénéité des stacks rend la mesure et l'optimisation globales difficiles.

6. Audit par critères

6.1 Maintenabilité

Évaluation	Constats	Lecture d'audit
Non validée sur le socle historique ; partielle sur UK/CA/US.	Code copié/adapté sur FR/DE/ES/IT, déploiements manuels, modèles divergents, stabilisation post-release de 3,4 jours sur OVH.	L'existant augmente le coût de correction et rend les règles métier difficiles à maintenir de manière uniforme. Les applications récentes améliorent certains aspects mais restent isolées.

6.2 Performance

Évaluation	Constats	Lecture d'audit
Partiellement validée, insuffisante pour un service international unifié.	Capacité de 150 req/s sur FR/DE/ES/IT contre 250 à 350 req/s sur UK/CA/US ; erreurs saisonnières jusqu'à 4 % sur le socle historique.	L'existant prouve que certains pays supportent mieux la charge, mais les écarts empêchent d'en faire un socle homogène pour les parcours critiques.

6.3 Évolutivité

Évaluation	Constats	Lecture d'audit
Non validée.	Monolithes par pays, aucune API unifiée, bases séparées et schémas divergents, partage manuel ou inexistant.	L'ajout de pays ou de règles tend à produire une nouvelle variante locale au lieu d'un enrichissement maîtrisé d'un socle commun.

6.4 Sécurité et confidentialité

Évaluation	Constats	Lecture d'audit
Non validée globalement ; meilleure posture sur CA/US.	SHA-1 sur FR/DE/ES/IT, TLS 1.0 encore présent sur FR/IT, secrets en fichiers de configuration, vulnérabilités de 35 à 41 % sur le socle historique.	L'hétérogénéité de sécurité empêche une garantie uniforme sur les comptes, les données personnelles et les intégrations. Les pratiques modernes restent ponctuelles.

6.5 Disponibilité et résilience

Évaluation	Constats	Lecture d'audit
Partiellement validée, mais insuffisante pour les parcours critiques.	Disponibilité de 97,2 % à 98,9 %, MTTR plus long sur OVH, absence de réplication FR/DE/ES/IT, sauvegardes non testées ou tests non réguliers.	L'existant peut fonctionner en régime courant, mais ne garantit pas une reprise robuste après incident ni une continuité homogène sur tous les pays.

6.6 Interopérabilité, API et données

Évaluation	Constats	Lecture d'audit
Non validée.	APIs limitées, non unifiées, données séparées par pays, échanges manuels ou inexistants.	L'existant ne répond pas à l'exigence d'une API agence CRUD sécurisée et alignée sur un modèle métier commun.

7. Impacts pour la proposition d'architecture

Cette section exprime les impacts à intégrer dans la proposition d'architecture, sans décrire encore la solution cible.

Impact issu de l'audit	Pourquoi c'est structurant	Critères concernés
Ne pas reprendre tel quel le découpage par pays.	Il produit duplication, divergences fonctionnelles et données incohérentes.	Maintenabilité, évolutivité, données.
Traiter la sécurité comme une contrainte transversale.	Les pratiques actuelles varient fortement selon les pays et certaines sont obsolètes.	Sécurité, conformité, exploitation.
Rendre les règles locales identifiables et gouvernables.	L'existant contient de la connaissance métier locale, mais elle est enfermée dans les variantes applicatives.	Internationalisation, maintenabilité.
Considérer les métriques de charge et d'erreur comme des seuils d'alerte.	Les pics saisonniers sont un risque directement lié à la perte de réservations.	Performance, disponibilité.
Prendre en compte les applications d'agence dès l'architecture.	Le besoin d'API CRUD sécurisée est central et l'existant ne le satisfait pas.	Interopérabilité, données, sécurité.
Prévoir une stratégie de reprise et de test de restauration.	Les sauvegardes non testées ou irrégulières rendent la reprise incertaine.	Résilience, disponibilité.

8. Conclusion de l'audit

L'existant reste utile pour comprendre les parcours de location et les règles propres à chaque pays.

En revanche, il ne constitue pas un socle technique commun : les applications, les données, les API et les pratiques de sécurité restent trop différentes.

Les principaux problèmes sont la duplication, les déploiements manuels, les pics de charge, la sécurité inégale et l'absence d'API et de données communes.

La future architecture doit donc conserver la connaissance métier tout en corrigeant ces limites techniques.

9. Proposition d'architecture cible

Cette partie propose une architecture cible pour la nouvelle application internationale Your Car Your Way. Le but est simple : remplacer les applications séparées par un socle commun, plus facile à maintenir et à sécuriser.

9.1 Objectifs techniques déduits de l'audit

Constat d'audit	Objectif simple	Choix prévu
Applications différentes selon les pays	Avoir un même parcours client partout	Une application Angular commune et une API Spring Boot centrale.
Bases de données séparées	Utiliser un modèle de données commun	Une base MySQL commune pour les comptes, agences, véhicules, offres, réservations, paiements et conversations support.
API limitées ou différentes	Proposer une API claire pour le web et les agences	API REST JSON, documentation Swagger, pagination et droits par rôle.
Sécurité inégale selon les pays	Moderniser la connexion et protéger l'API	Spring Security, mots de passe hachés avec BCrypt, JWT signé, HTTPS, secrets hors code et stack ELK pour les logs, la recherche et les alertes.
Pics saisonniers	Éviter les traitements trop lourds pendant la réservation	Transactions courtes, index, cache simple et suivi des paiements.
Sauvegardes et restaurations peu testées	Pouvoir restaurer les données en cas d'incident	Base managée, sauvegardes automatiques et tests de restauration réguliers.
Contraintes RGAA, RGPD et sobriété numérique	Les intégrer dès le début	Écrans accessibles, données limitées au nécessaire et pages légères.

9.2 Décisions structurantes

Décision	Choix retenu	Pourquoi ce choix
Style d'architecture	Architecture en couches	C'est le choix principal. Angular gère l'affichage, Spring Boot expose l'API, les services portent les règles métier et MySQL stocke les données.
Front-end	Angular	Angular est adapté pour une application web internationale, responsive et accessible.
Back-end	Spring Boot	Spring Boot permet de créer une API REST claire, sécurisée et facile à tester.
Base de données	MySQL	Le besoin repose sur des données relationnelles classiques. MySQL couvre les besoins de la V1 sans complexité supplémentaire.
Authentification	Spring Security + BCrypt + JWT signé	Les mots de passe sont hachés avec BCrypt. Après connexion, l'utilisateur reçoit un JWT signé ; l'API vérifie le jeton et les rôles avant chaque action sensible.
Paiement	PSP externe type Stripe	Les données de carte bancaire restent

		chez le prestataire de paiement. L'application ne stocke que les références utiles.
Documentation API	Swagger	Les applications d'agence disposent d'un contrat d'API lisible et stable.
Déploiement cible	Conteneurs applicatifs, base managée et pipeline CI/CD	Le pipeline CI/CD automatise le build, les tests et le déploiement sans alourdir la V1.

9.3 Options comparées

Le tableau compare les différentes architectures. Pour ce projet, le choix retenu est l'architecture en couches.

Architecture	Intérêt pour le projet	Limite principale	Décision
Architecture client-serveur	Le navigateur et les applications d'agence appellent un serveur central. C'est simple à comprendre et adapté à une API REST.	Ce nom décrit surtout la communication. Il ne dit pas comment organiser le code interne.	Utilisée comme mode de communication, mais pas comme style principal.
Architecture en couches	Elle sépare l'affichage, l'API, les règles métier et la base de données. Les responsabilités sont plus claires.	Il faut éviter d'ajouter trop de couches inutiles.	Retenue comme architecture cible.
Architecture microservices	Chaque domaine peut devenir un service séparé et évoluer de son côté.	Trop complexe pour une première version : plus de déploiements, plus de supervision et plus de risques.	Non retenue.
Architecture pilotée par les événements	Utile pour envoyer des notifications, traiter des webhooks ou lancer des actions en arrière-plan.	Trop lourde si elle devient le cœur de tout le système.	Non retenue.
Architecture orientée services	Elle expose des services/API consommables par d'autres applications, par exemple les applications d'agence.	Elle ne suffit pas à organiser tout le code et la base de données.	Non retenue.
Architecture modulaire	Elle aide à ranger le code par domaines : profil, agences, offres, réservation, paiement et support.	Si les modules sont mal séparés, le code peut redevenir confus.	Utilisée comme principe interne, mais pas comme architecture principale.
Architecture centrée sur les données	Elle permet de gérer des règles par données : pays, devises, catégories ACRIS et règles locales.	Si les données de paramétrage sont mauvaises, le comportement métier devient mauvais aussi.	Non retenue.
Reprise des applications par pays	Elle limite le changement au départ et réutilise l'existant.	Elle garde les mêmes problèmes : duplication, sécurité inégale et bases différentes.	Rejetée.

Conclusion : l'architecture en couches est le choix le plus adapté.

10. Spécifications techniques alignées avec les user stories

Les spécifications suivantes traduisent les user stories en choix techniques simples. Elles indiquent ce que l'application doit faire et comment le faire sans complexifier l'architecture.

10.1 Traçabilité fonctionnel -> technique

User stories	Besoin métier	Réponse technique
US-01 à US-06, US-29	Compte, connexion, profil, suppression et droits RGPD	Module Identité & Profil. Mots de passe hachés avec BCrypt, connexion avec JWT signé, profil en base, réinitialisation sécurisée, consentements tracés et suppression/anonymisation contrôlée.
US-07 à US-12	Agences, flotte, recherche et détail d'offre	Module Agences, Flotte & Offres. API de recherche paginée, filtres par pays/ville/dates/catégorie, référentiel ACRIS et disponibilité calculée à partir des véhicules et réservations.
US-13 à US-19	Réservation, modification, annulation, remboursement	Module Réservations. Le client réserve une catégorie ACRIS ; un véhicule précis peut être affecté plus tard par l'agence. Statuts simples, règle des 48 h et calcul du remboursement. Les opérations importantes sont journalisées dans ELK.
US-15, US-16, US-20, US-21	Paiement, confirmation, reçus, notifications	Module Paiement & Notifications. Prestataire de paiement externe, webhooks signés, reçus stockés et e-mails transactionnels.
US-22	Assistance client	Module Support & Tchat dans la cible : conversations et historique en MySQL, nouveaux messages diffusés par WebSocket. Le PoC valide uniquement le flux temps réel et l'historique temporaire avec Angular, Spring Boot et H2.
US-23 à US-25	Accessibilité PSH	Composants accessibles, navigation clavier, focus visible, libellés et messages d'erreur compréhensibles.
US-26 à US-28	API agence et conformité	API REST protégée par JWT et rôles, documentée avec Swagger. Les logs sont centralisés dans ELK sans données sensibles.
US-30	Sobriété numérique	Chargement progressif, pagination, cache des référentiels et pages plus légères.

10.2 Spécifications front-end Angular

- Angular est organisé par domaines : identité/profil, agences, recherche, offres, réservation, paiement, historique et assistance. Le contact avec le support est matérialisé par le PoC de tchat en temps réel.
- Les pages publiques sont séparées des pages protégées. Les pages protégées vérifient que l'utilisateur est connecté et possède le bon rôle.
- Les formulaires utilisent des validations côté client avec des messages d'erreur simples.
- Les libellés, dates, heures, devises et messages sont internationalisés. Les règles métier restent côté back-end.
- Angular appelle l'API avec HttpClient. Quand l'utilisateur est connecté, le JWT est envoyé dans l'en-tête Authorization: Bearer.
- L'accessibilité est prévue dès le début : titres clairs, navigation clavier, focus visible et erreurs reliées aux champs.
- Le front reste léger : lazy loading, images optimisées, listes paginées et suppression des éléments trop lourds.

10.3 Spécifications back-end Spring Boot

- L'API REST JSON est versionnée sous /api/v1. Les réponses d'erreur gardent le même format.
- Le back-end est découpé par grands domaines : profil, agences, flotte de véhicules, offres, réservations, paiement, documents et support. Les conversations sont conservées dans MySQL et les nouveaux messages passent par WebSocket. La PoC reste volontairement plus simple avec H2.
- Spring Security gère l'authentification : les mots de passe sont hachés avec BCrypt, les liens de réinitialisation sont courts, à usage unique et limités dans le temps, puis le JWT signé et les rôles client, agence, support, conformité et administration technique sont vérifiés.
- Les règles métier sont appliquées côté back-end : 48 h, 7 jours/25 %, statuts de réservation, paiement et remboursement.
- MySQL stocke les données métier communes. Les évolutions du schéma sont versionnées.
- Les paiements et remboursements sont protégés contre les doubles traitements grâce à une clé d'idempotence unique et au suivi de la référence du PSP.
- Les logs applicatifs et les opérations sensibles sont centralisés dans ELK pour la recherche et les alertes.
- La sécurité de base est appliquée : validation des entrées, CORS limité, secrets hors code, HTTPS.

10.4 Spécifications API agence

Domaine	Endpoints indicatifs	Règles simples
Utilisateurs / profils	GET/POST/PATCH /api/v1/users ; GET/PATCH /api/v1/profiles/{id}	Données sensibles masquées selon le rôle ; suppression ou anonymisation contrôlée.
Agences	GET/POST/PATCH /api/v1/agencies ; GET /api/v1/agencies/{id}	Filtres par pays, ville et statut ; pagination obligatoire.
Catégories ACRIS	GET /api/v1/vehicle-categories	Référentiel lisible ; mise à jour réservée aux rôles autorisés.
Véhicules	GET/POST/PATCH /api/v1/vehicles ; GET /api/v1/vehicles/{id}	Gestion de la flotte par agence et catégorie ACRIS ; affectation à une réservation réservée aux rôles autorisés.
Offres / disponibilités	GET /api/v1/offers ; GET /api/v1/offers/{id}	Offres calculées ou temporaires ; disponibilité vérifiée par catégorie, agence et période. Le client ne choisit jamais un

		véhicule précis.
Réservations	GET/POST/PATCH /api/v1/reservations ; POST /cancel ; POST /change-status	Règles métier vérifiées côté serveur ; protection contre les doubles traitements.
Paielements / remboursements	GET /api/v1/payments/{id} ; POST /api/v1/refunds	Aucune donnée carte ; suivi du PSP ; opérations journalisées dans la stack ELK.
Documents	GET /api/v1/documents/{id}	Contrôle des droits ; lien temporaire ou téléchargement contrôlé.

10.5 Exigences non fonctionnelles

Les KPI ci-dessous restent volontairement limités aux points essentiels de la V1.

Axe	Exigence cible mesurable	Mécanisme de mesure / preuve
Disponibilité	Disponibilité mensuelle $\geq 99,5$ %.	Supervision mensuelle.
MTTR	Temps moyen de rétablissement ≤ 1 h 00.	Registre des incidents.
Déploiements	Taux de réussite des déploiements ≥ 95 %.	Historique du pipeline CI/CD.
Stabilisation après mise en production	Retour à une situation stable en moins de 1 jour.	Suivi des incidents après déploiement.
Performance sous forte charge	Taux d'erreur < 1 % lors d'un test à 350 req/s.	Rapport de test de charge.
Lighthouse	Scores performance et accessibilité ≥ 90 .	Rapport Lighthouse.
SonarCloud	Quality Gate validé.	Rapport SonarCloud.

11. Architecture applicative cible

L'architecture cible est en couches : Angular pour l'interface, Spring Boot pour l'API et le WebSocket, MySQL pour les données et ELK pour les logs. Le paiement, l'e-mail et le stockage documentaire restent externalisés.

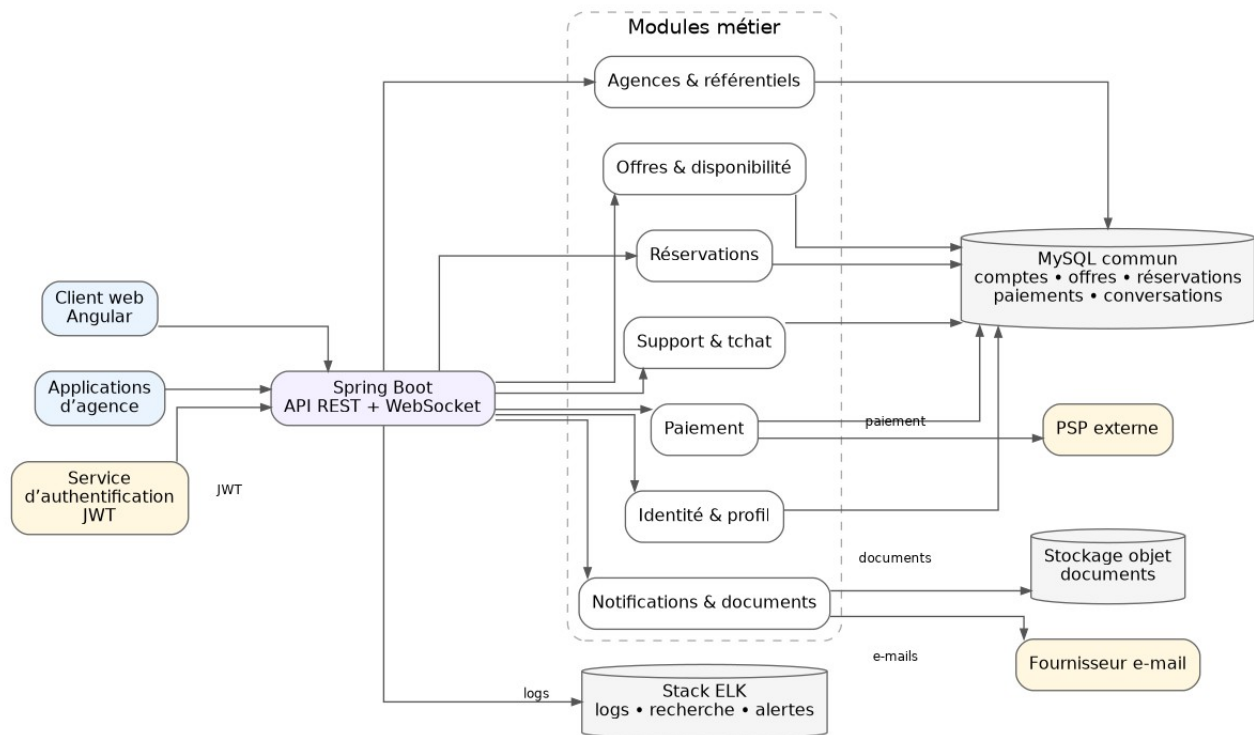


Figure 2 - Diagramme simplifié des composants de l'architecture cible.

11.1 Principes de découpage applicatif

Module	Responsabilité	Dépendances autorisées
Identity & Profile	Compte, profil client, consentements et suppression/anonymisation.	JWT et MySQL.
Agency, Fleet & Reference	Agences, véhicules, pays, villes, horaires, catégories ACRISS et règles locales.	Base référentiels, cache simple.
Offer & Availability	Recherche d'offres, prix, disponibilités et conditions affichées.	Agences, véhicules, catégories, réservations existantes et règles locales.
Booking	Création, modification, annulation, statuts et réservation d'une catégorie ; affectation optionnelle d'un véhicule physique.	Offres, profil, catégories, véhicules et paiement.
Payment & Refund	Paiement, webhook, remboursement et rapprochement.	PSP et réservation.
Notification & Document	Confirmation, reçu, facture et e-mail transactionnel.	Réservation, paiement, stockage document, fournisseur e-mail.
Support	Contact client-support en temps réel et historique de conversation.	WebSocket, profil et réservation si besoin.

11.2 Diagramme de déploiement cible

Le déploiement reste simple : Angular est servi en statique, Spring Boot tourne en conteneur, MySQL peut être managé et les logs sont envoyés vers ELK. Le pipeline CI/CD automatise les tests et le déploiement.

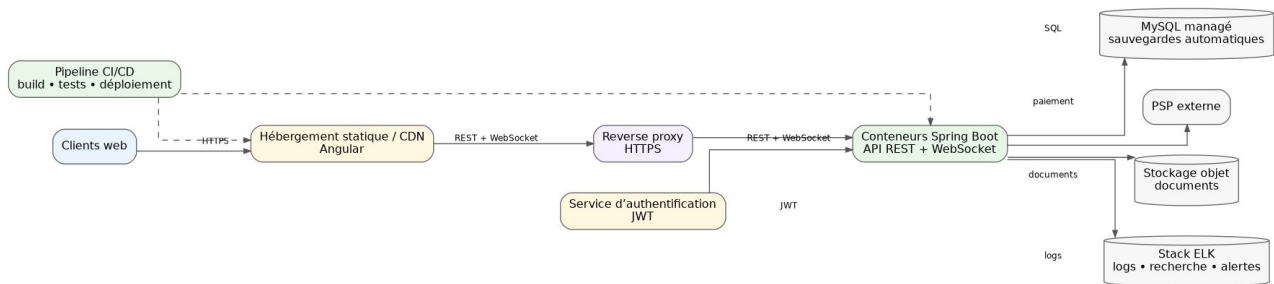


Figure 3 - Diagramme simplifié de déploiement cible.

11.3 Flux principaux

Flux	Étapes principales	Contrôles
Recherche d'offre	Angular appelle GET /offers avec les critères. Spring Boot valide la recherche et calcule les offres à partir des agences, catégories, véhicules disponibles, réservations existantes et tarifs.	Dates et fuseaux vérifiés, chevauchements de réservation contrôlés, pagination, cache des référentiels et connexion non obligatoire.
Réservation	Le client connecté choisit une offre. Spring Boot réserve la catégorie ACRISS, copie les informations utiles dans la réservation, crée le statut PENDING_PAYMENT puis lance le paiement.	Disponibilité de la catégorie vérifiée, prix recalculé côté back-end et transaction courte. Le véhicule précis reste facultatif et peut être affecté plus tard par l'agence.
Paiement	Spring Boot crée l'intention de paiement. Le client paie via le PSP. Le webhook confirme ou refuse le paiement.	Aucune donnée carte, webhook signé et protection contre les doubles traitements.
Modification	Le client ou l'agence demande une modification. Spring Boot applique la règle des 48 h.	Règle vérifiée côté serveur et prix recalculé si besoin.
Annulation	Spring Boot calcule le remboursement. Le client confirme. Le PSP rembourse si c'est autorisé.	Montant affiché avant confirmation, règle 7 jours/25 % et protection contre les doubles traitements.
API agence	L'application agence reçoit un JWT technique et appelle uniquement les endpoints autorisés.	Rôles dédiés, Swagger et masquage des données sensibles.
Tchat support	Angular charge l'historique d'une conversation par REST puis échange les nouveaux messages avec Spring Boot par WebSocket.	Canal bidirectionnel, conversation isolée par identifiant, validation côté serveur et historique conservé. La PoC ne valide ni l'authentification ni le routage multi-conversations.

11.4 Architecture de la preuve de concept

La PoC valide uniquement le tchat support : historique par REST, messages en temps réel par WebSocket et stockage temporaire dans H2.

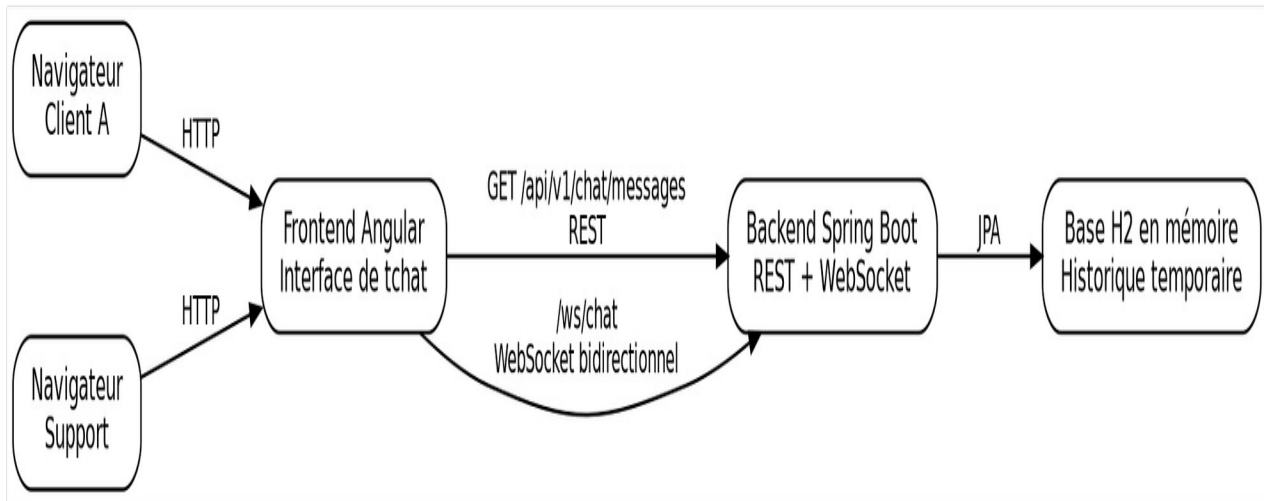


Figure 4 - Architecture de la preuve de concept de tchat support.

Le schéma montre les deux navigateurs, Angular, Spring Boot, REST, WebSocket et H2. H2 sert uniquement à la démonstration.

12. Modèle de données

Le modèle de données commun reste volontairement limité à neuf tables. Il couvre les comptes, les agences, les catégories ACRISS, les véhicules physiques, les offres temporaires, les réservations, les paiements et le tchat support. Le client réserve une catégorie de véhicule ; un véhicule précis peut seulement être affecté plus tard par l'agence. Les logs restent dans ELK et les documents dans le stockage objet.

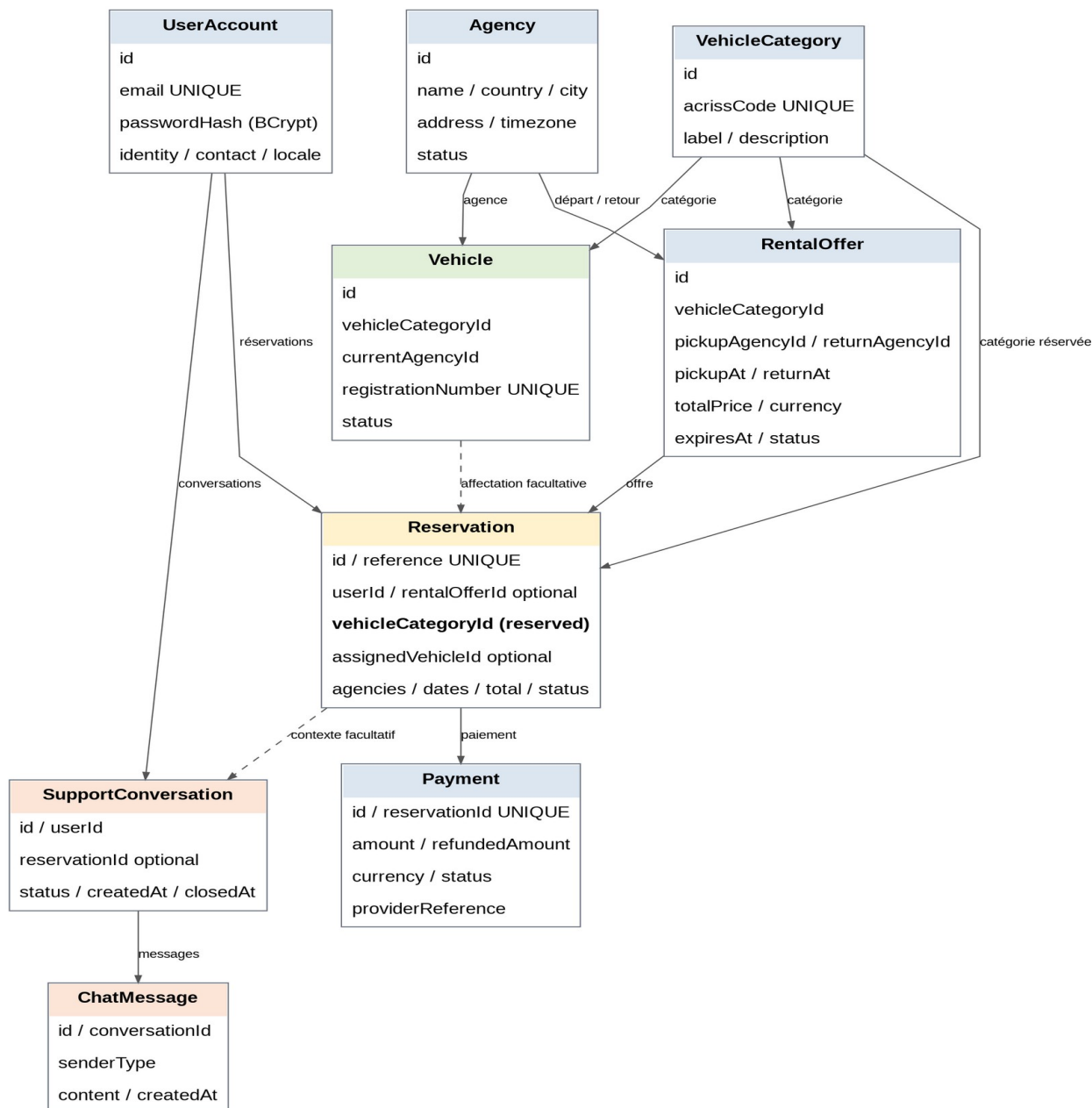


Figure 5 - Modèle de données.

12.1 Entités principales

Entité	Rôle métier	Relations clés / contraintes
UserAccount	Compte et profil client.	E-mail unique, mot de passe haché avec BCrypt, identité, contact, locale et statut.
Agency	Agence de départ ou de retour.	Nom, pays, ville, adresse, fuseau horaire et statut.
VehicleCategory	Catégorie de véhicule réservée par le client.	Code ACRISS unique, libellé et description.
Vehicle	Véhicule physique de la flotte.	Catégorie, agence courante, immatriculation unique et statut. Il n'est jamais choisi directement par le client.
RentalOffer	Offre temporaire issue d'une	Catégorie, agences, dates, prix,

	recherche.	devise, statut et date d'expiration. Aucun véhicule précis.
Reservation	Réservation confirmée ou en cours.	Client, offre d'origine facultative, catégorie réservée, véhicule affecté facultatif, agences, dates, total et statut.
Payment	Paielement lié à une réservation.	Montant, montant remboursé, devise, statut et référence du PSP. Une table séparée de remboursement n'est pas nécessaire.
SupportConversation	Conversation entre un client et le support.	Liée au client et facultativement à une réservation ; statut, ouverture et clôture.
ChatMessage	Message d'une conversation support.	Conversation, type d'émetteur, contenu et date ; historique ordonné chronologiquement.

12.2 Règles de modélisation des données

- Les données de carte bancaire ne sont jamais stockées. Seules les références du prestataire de paiement sont conservées.
- RentalOffer représente un résultat de recherche temporaire : catégorie, agences, dates, prix et expiration. L'offre ne pointe jamais vers un véhicule précis.
- Une réservation porte toujours sur une catégorie ACRIS. assignedVehicleId reste facultatif et peut être renseigné plus tard par l'agence.
- La disponibilité est contrôlée par catégorie, agence et période à partir des véhicules actifs et des réservations qui se chevauchent.
- La réservation conserve les données essentielles de l'offre validée afin que l'historique reste correct après l'expiration de RentalOffer.
- Payment couvre le paiement et le remboursement avec amount et refundedAmount ; aucune table Refund séparée n'est nécessaire.
- Une SupportConversation peut être liée à une réservation. Les ChatMessage sont rattachés à la conversation et ordonnés par date.
- Les mots de passe sont hachés avec BCrypt, les données sensibles sont limitées au nécessaire et les évolutions du schéma MySQL sont versionnées.

12.3 Script de création MySQL

Le script ci-dessous crée directement les neuf tables du modèle. Il correspond au diagramme présenté ci-dessus.

```
CREATE DATABASE IF NOT EXISTS ycyw;
USE ycyw;

CREATE TABLE user_account (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(60) NOT NULL,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  phone VARCHAR(30),
```

```

locale VARCHAR(10) NOT NULL DEFAULT 'fr-FR',
status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
terms_accepted_at DATETIME,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE agency (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(150) NOT NULL,
country_code CHAR(2) NOT NULL,
city VARCHAR(100) NOT NULL,
address VARCHAR(255) NOT NULL,
timezone VARCHAR(50) NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
);

CREATE TABLE vehicle_category (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
acriiss_code VARCHAR(4) NOT NULL UNIQUE,
label VARCHAR(100) NOT NULL,
description VARCHAR(255)
);

CREATE TABLE vehicle (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
vehicle_category_id BIGINT NOT NULL,
current_agency_id BIGINT NOT NULL,
registration_number VARCHAR(30) NOT NULL UNIQUE,
status VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE',
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_vehicle_category
FOREIGN KEY (vehicle_category_id)
REFERENCES vehicle_category(id),
CONSTRAINT fk_vehicle_agency
FOREIGN KEY (current_agency_id)
REFERENCES agency(id)
);

CREATE TABLE rental_offer (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
vehicle_category_id BIGINT NOT NULL,
pickup_agency_id BIGINT NOT NULL,
return_agency_id BIGINT NOT NULL,
pickup_at DATETIME NOT NULL,
return_at DATETIME NOT NULL,
total_price DECIMAL(10, 2) NOT NULL,
currency CHAR(3) NOT NULL,
expires_at DATETIME NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE',
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT chk_offer_dates
CHECK (return_at > pickup_at),
CONSTRAINT chk_offer_price
CHECK (total_price >= 0),
CONSTRAINT fk_offer_category
FOREIGN KEY (vehicle_category_id)
REFERENCES vehicle_category(id),
CONSTRAINT fk_offer_pickup_agency
FOREIGN KEY (pickup_agency_id)
REFERENCES agency(id),
CONSTRAINT fk_offer_return_agency
FOREIGN KEY (return_agency_id)
REFERENCES agency(id)
);

CREATE TABLE reservation (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
reference VARCHAR(30) NOT NULL UNIQUE,
user_id BIGINT NOT NULL,
rental_offer_id BIGINT,
vehicle_category_id BIGINT NOT NULL,
assigned_vehicle_id BIGINT,
pickup_agency_id BIGINT NOT NULL,
return_agency_id BIGINT NOT NULL,
pickup_at DATETIME NOT NULL,
return_at DATETIME NOT NULL,
total_price DECIMAL(10, 2) NOT NULL,
currency CHAR(3) NOT NULL,
status VARCHAR(30) NOT NULL DEFAULT 'PENDING_PAYMENT',
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT chk_reservation_dates
CHECK (return_at > pickup_at),
CONSTRAINT chk_reservation_price
CHECK (total_price >= 0),
CONSTRAINT fk_reservation_user
FOREIGN KEY (user_id)

```

```

REFERENCES user_account(id),
CONSTRAINT fk_reservation_offer
FOREIGN KEY (rental_offer_id)
REFERENCES rental_offer(id)
ON DELETE SET NULL,
CONSTRAINT fk_reservation_category
FOREIGN KEY (vehicle_category_id)
REFERENCES vehicle_category(id),
CONSTRAINT fk_reservation_vehicle
FOREIGN KEY (assigned_vehicle_id)
REFERENCES vehicle(id)
ON DELETE SET NULL,
CONSTRAINT fk_reservation_pickup_agency
FOREIGN KEY (pickup_agency_id)
REFERENCES agency(id),
CONSTRAINT fk_reservation_return_agency
FOREIGN KEY (return_agency_id)
REFERENCES agency(id)
);

CREATE TABLE payment (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
reservation_id BIGINT NOT NULL UNIQUE,
amount DECIMAL(10, 2) NOT NULL,
refunded_amount DECIMAL(10, 2) NOT NULL DEFAULT 0,
currency CHAR(3) NOT NULL,
status VARCHAR(30) NOT NULL DEFAULT 'PENDING',
provider_reference VARCHAR(100) UNIQUE,
paid_at DATETIME,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT chk_payment_amount
CHECK (amount >= 0),
CONSTRAINT chk_refunded_amount
CHECK (refunded_amount >= 0 AND refunded_amount <= amount),
CONSTRAINT fk_payment_reservation
FOREIGN KEY (reservation_id)
REFERENCES reservation(id)
);

CREATE TABLE support_conversation (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
user_id BIGINT NOT NULL,
reservation_id BIGINT,
status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
closed_at DATETIME,
CONSTRAINT fk_conversation_user
FOREIGN KEY (user_id)
REFERENCES user_account(id),
CONSTRAINT fk_conversation_reservation
FOREIGN KEY (reservation_id)
REFERENCES reservation(id)
ON DELETE SET NULL
);

CREATE TABLE chat_message (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
conversation_id BIGINT NOT NULL,
sender_type VARCHAR(20) NOT NULL,
content VARCHAR(1000) NOT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_message_conversation
FOREIGN KEY (conversation_id)
REFERENCES support_conversation(id)
ON DELETE CASCADE
);

CREATE INDEX idx_vehicle_availability
ON vehicle (vehicle_category_id, current_agency_id, status);

CREATE INDEX idx_offer_search
ON rental_offer (
vehicle_category_id,
pickup_agency_id,
pickup_at,
return_at,
status
);

CREATE INDEX idx_reservation_availability
ON reservation (
vehicle_category_id,
pickup_agency_id,
pickup_at,
return_at,
status
);

```

```
);  
  
CREATE INDEX idx_reservation_user  
ON reservation (user_id, created_at);  
  
CREATE INDEX idx_chat_history  
ON chat_message (conversation_id, created_at);
```

12.4 Structure de données de la PoC

La PoC utilise une seule entité ChatMessage. Elle valide le flux technique, pas la gestion complète des conversations support.

Attribut	Type / contrainte	Rôle
id	Long, clé technique auto-générée	Identifie un message et permet d'ordonner l'historique.
author	Chaîne obligatoire, 80 caractères maximum	Pseudo de l'émetteur : par exemple Client A ou Support.
content	Chaîne obligatoire, 1 000 caractères maximum	Contenu du message de démonstration.
createdAt	Instant obligatoire	Date et heure de création du message.

H2 stocke l'historique temporaire de la PoC. Aucune donnée sensible n'y est enregistrée. En production, chaque message sera rattaché à une conversation et à un émetteur.

13. Intégration des composants tiers

Les intégrations et services techniques sont utilisés seulement quand ils réduisent le risque : authentification Spring Security avec BCrypt et JWT, paiement, e-mail, stockage documentaire et observabilité via stack ELK. Chaque intégration doit rester simple, sécurisée et testable.

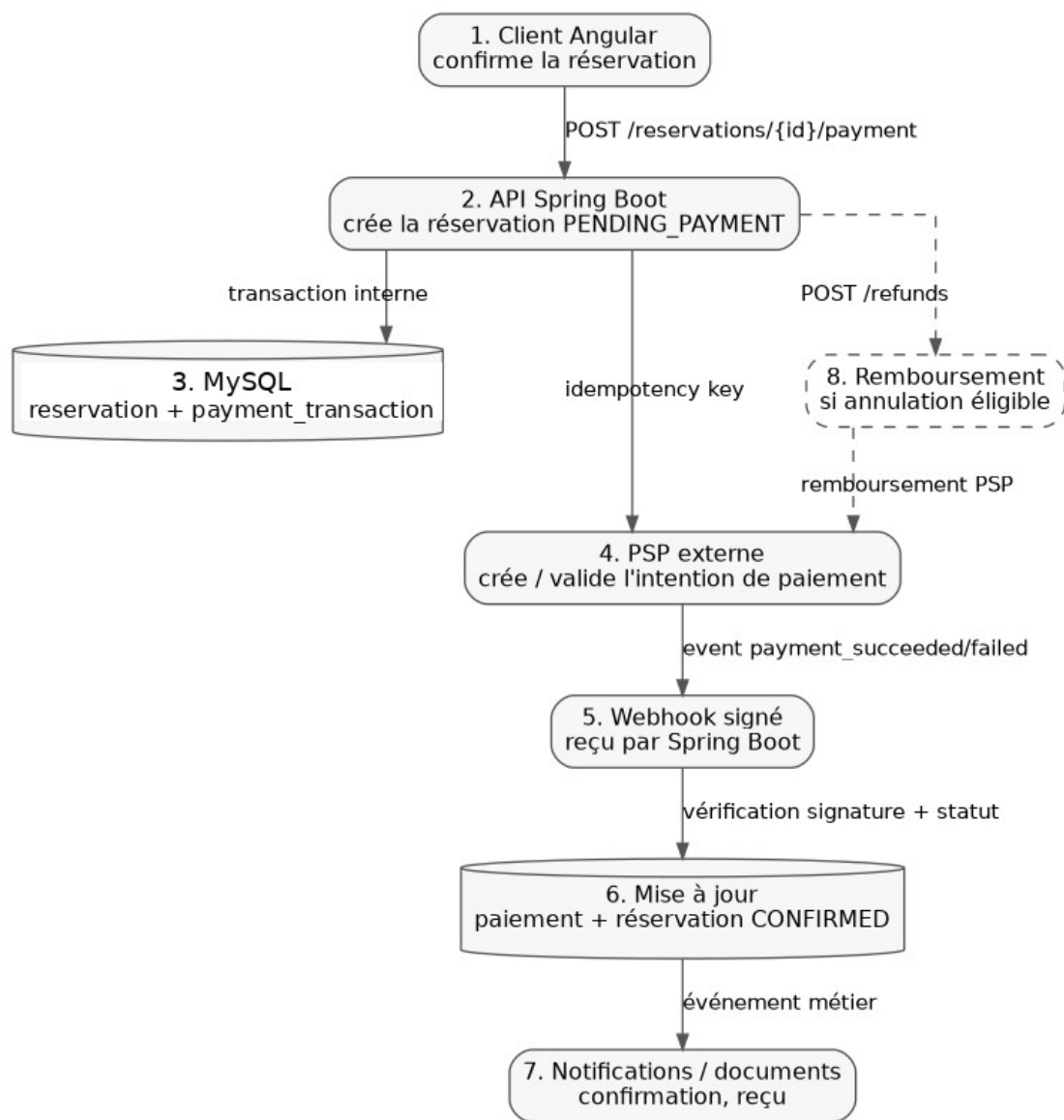


Figure 6 - Flux simplifié d'intégration PSP.

13.1 Plan d'intégration

Composant / service	Rôle dans l'architecture	Mode d'intégration	Points de vigilance
Authentification Spring Security / JWT	Connexion client, hashage des mots de passe, rôles agence/support/conformité et émission de jetons.	Les mots de passe sont hachés avec BCrypt. Après connexion, le client reçoit un JWT signé ; Spring Boot vérifie la signature, l'expiration et les rôles.	Coût BCrypt à calibrer, durée de vie du jeton, renouvellement, stockage côté front et révocation en cas de problème.
PSP : Stripe ou équivalent	Paieement et remboursement externalisés.	Création du paiement côté back-end, webhook signé et référence PSP stockée.	Aucune carte en base, vérification du webhook, retries, rapprochement et messages d'erreur clairs.
Fournisseur e-mail	Notifications transactionnelles : confirmation, annulation, modification et remboursement.	Appel serveur à serveur, modèles traduits et relance possible en cas d'échec.	Limiter les données personnelles et suivre les erreurs d'envoi.

Stockage objet	Conservation des reçus, factures et confirmations.	Upload depuis le back-end, puis URL temporaire ou téléchargement contrôlé.	Droits d'accès, durée de conservation, chiffrement et suppression.
Observabilité	Stack ELK pour logs, recherche, alertes et tableaux de bord.	Les logs sont envoyés vers ELK pour la recherche et les alertes. SonarCloud contrôle la qualité du code ; Lighthouse vérifie la performance et l'accessibilité du frontend.	Ne pas journaliser les cartes, mots de passe, JWT ou données personnelles inutiles.
Référentiel ACRIS	Normalisation des catégories véhicule.	Import initial ou table maintenue, avec libellés traduits.	Conserver les codes officiels et éviter les catégories locales incompatibles.

13.2 Stratégie d'intégration PSP

- Le back-end crée d'abord une réservation au statut PENDING_PAYMENT.
- Il protège le traitement contre la création de deux paiements en cas de nouvel essai.
- Le front ne traite jamais les données de carte. Il utilise le composant sécurisé ou la redirection du PSP.
- Le webhook PSP confirme le paiement. C'est lui qui fait passer la réservation en CONFIRMED.
- Un job de réconciliation vérifie les réservations restées en PENDING_PAYMENT.
- Les remboursements sont calculés par Spring Boot puis envoyés au PSP.

13.3 Stratégie JWT et API agence

- Les clients se connectent puis reçoivent un JWT signé.
- Les applications d'agence utilisent un JWT technique avec des droits limités.
- Spring Boot ne fait pas confiance au front : il vérifie le JWT, le rôle et les règles métier à chaque endpoint sensible.
- Les rôles cibles sont : CLIENT, AGENCY_APP, SUPPORT, COMPLIANCE et ADMIN_TECH.
- Les JWT, mots de passe et secrets ne sont jamais stockés dans le code source. Les secrets applicatifs viennent de l'environnement ou d'un coffre de secrets.

14. Justification des choix technologiques

14.1 Comparaison des solutions technologiques

Les choix structurants ont été comparés à des alternatives selon quatre critères prioritaires : simplicité, maintenabilité, compatibilité avec l'existant et adéquation au besoin métier.

Besoin	Solution retenue	Alternatives examinées	Décision et critères
Front web international	Angular	React, Vue	Retenu pour la cohérence avec l'expérimentation Angular/Spring Boot existante, la structuration des formulaires et du routage, les tests et l'accessibilité. React reste viable mais prolongerait la diversité déjà présente dans le SI.
Backend API	Spring Boot	Node.js, Laravel	Retenu pour la cohérence avec l'expérience Java de l'entreprise, les transactions, Spring Security et la

			maintenabilité. Node.js et Laravel sont adaptés à d'autres contextes mais ajouteraient une stack supplémentaire.
Base de données principale	MySQL	PostgreSQL ; base NoSQL	MySQL est retenu car le projet repose surtout sur des données relationnelles classiques : comptes, agences, offres, réservations et paiements. PostgreSQL conviendrait aussi, mais n'apporte pas d'avantage déterminant. MySQL suffit donc au besoin tout en gardant une architecture simple.
Tchat support temps réel	WebSocket	Polling HTTP	Retenu car l'échange est bidirectionnel. Le polling multiplie les requêtes inutiles. La PoC valide ce choix.

14.2 Justification et points de vigilance

Technologie	Apport pour le projet	Limites / vigilance
Angular	Répond au besoin front : application internationale, formulaires, routage, accessibilité.	Surveiller le poids du bundle, le lazy loading et les tests d'accessibilité.
Spring Boot	Permet de créer rapidement une API REST sécurisée, testable et observable.	Garder des modules clairs pour éviter un code trop mélangé.
Spring Security	Vérifie le JWT, protège les endpoints et contrôle les rôles.	Tester précisément CORS, endpoints publics/protégés et règles d'autorisation.
MySQL	Adapté aux transactions, contraintes d'intégrité, réservations et paiements.	Limiter le modèle aux tables durables utiles ; surveiller les index et éviter de stocker en colonnes JSON ce qui porte des contraintes métier.
Swagger	Donne un contrat lisible pour les applications d'agence.	Les exemples métier doivent être relus et complétés.
JWT signé	Transporte l'identité et les rôles de l'utilisateur entre le front et l'API.	Prévoir expiration, renouvellement, stockage côté front et révocation.
Stripe ou PSP équivalent	Externalise les données carte, les webhooks et les remboursements.	Prévoir les erreurs, les nouveaux essais, les doubles paiements et le mode de test.
Docker / conteneurs + pipeline CI/CD	Rend les builds, tests et déploiements reproductibles et facilite le passage entre environnements.	Faire attention aux versions des images utilisées et bloquer les déploiements si les tests ou contrôles qualité échouent.

BCrypt

Hache les mots de passe avec un algorithme adaptatif intégré à Spring Security.

Calibrer le coût sur l'environnement cible et ne jamais journaliser ni stocker le mot de passe brut.

15. Sécurité, accessibilité et impact écologique

15.1 Sécurité et conformité

- HTTPS partout ; suppression des protocoles obsolètes constatés dans l'existant.
- API protégées par JWT et rôles, avec vérification des droits côté service.
- Mots de passe hachés avec BCrypt via Spring Security ; aucun mot de passe en clair et aucun ancien hash SHA-1 repris dans la nouvelle application. Le coût est calibré sur l'environnement cible.
- Validation des entrées côté front et back ; messages d'erreur utiles sans fuite d'information sensible.
- Les opérations sensibles sont journalisées dans ELK : création, modification, annulation, paiement, remboursement et suppression/anonymisation.
- Secrets stockés hors code et séparation claire entre dev, test et production.
- RGPD : minimisation, information client, consentements, durées de conservation et suppression/anonymisation.

15.2 Accessibilité

- Les parcours sont conçus et testés au clavier : compte, connexion, recherche, réservation, paiement, modification, annulation et suppression.
- Tous les champs ont des libellés liés. Les erreurs sont compréhensibles et associées au bon champ.
- Les composants Angular ont un focus visible et un ordre de tabulation logique.
- Les contrastes, tailles de texte, zones cliquables et messages non uniquement visuels sont vérifiés.
- Les documents générés restent structurés et lisibles autant que possible.

15.3 Éco-conception

- Ne collecter que les données nécessaires à la location, au paiement, au support et aux obligations réglementaires.
- Paginer les offres, agences et historiques pour éviter les transferts inutiles.
- Charger les routes Angular progressivement et supprimer les médias non essentiels.
- Mettre en cache les référentiels peu changeants : pays, agences, catégories ACRIS et règles locales publiées.
- Contrôler régulièrement les pages critiques avec Lighthouse.

16. Synthèse finale

L'architecture cible remplace les applications par pays par un socle commun Angular, Spring Boot et MySQL. Les règles locales deviennent des paramètres métier. L'architecture en couches garde les responsabilités simples et séparées.

Les composants externes restent limités à l'authentification, au paiement, aux e-mails, au stockage documentaire et à ELK pour les logs. La PoC valide le tchat WebSocket avec H2 uniquement pour la démonstration. La sécurité, l'accessibilité, le RGPD et la sobriété restent pris en compte dès la conception.

Références techniques utilisées

- Angular - Internationalization : <https://angular.dev/guide/i18n>
- Angular - Accessibility best practices : <https://angular.dev/best-practices/a11y>
- Angular - HTTP Client : <https://angular.dev/guide/http>
- Spring Boot - Spring Security : <https://docs.spring.io/spring-boot/reference/web/spring-security.html>
- Spring Security - Password Storage / BCrypt :
<https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>
- Spring Security - JWT Resource Server :
<https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>
- Swagger : <https://springdoc.org/>
- RFC 7519 - JSON Web Token (JWT) : <https://www.rfc-editor.org/rfc/rfc7519>
- Stripe - Payment Intents : <https://docs.stripe.com/payments/payment-intents>
- Stripe - Webhooks : <https://docs.stripe.com/webhooks>
- OpenClassrooms - Définissez votre architecture logicielle grâce aux standards reconnus : architectures client-serveur, pilotée par les événements, orientée services, modulaire, en couches et centrée sur les données : <https://openclassrooms.com/fr/courses/7210131-definissez-votre-architecture-logicielle-grace-aux-standards-reconnus>